



# TypeScript dá forma aos dados do nosso app

Componentes, props, estado e coleções • Aula 4

---

Técnico em Informática Integrado ao Ensino Médio

Instituto Federal do Maranhão — IFMA

**Prof. Thales Levi Azevedo Valente**

**O ambiente funciona; agora o aplicativo precisa compreender seus dados.**



## Retrospectiva verificável da Aula 3

- ✓ `node` , `npm` e `git` respondendo no terminal
- ✓ `npm ci` reproduziu as dependências com sucesso
- ✓ `npx expo-doctor` terminou sem erros bloqueantes
- ✓ `npx expo start` abre no navegador e no celular
- ✓ O primeiro commit existe no histórico Git

# Sem tipos, um dado errado pode chegar até a tela

## JAVASCRIPT COMUM

```
const memoria = {  
  id: 1,  
  titulo: "Relato",  
  ano: "dois mil"  
};
```



### O que pode dar errado?

Imagine tentar calcular a idade deste registro ou formatar a data para exibição. O erro só apareceria quando o usuário abrisse o app.

# Resultados de Aprendizagem

Ao final desta aula, você será capaz de:



## MODELAR

Dados tipados e interfaces



## TRANSFORMAR

Coleções em componentes



## COMPOR

Interfaces reutilizáveis



## INTERAGIR

Usando estado e eventos



## TESTAR

Mudanças no navegador/celular



## EXPLICAR

A lógica do seu próprio código

# Agenda de 100 minutos

0 – 10 min



Retrospectiva verificável da Aula 3

10 – 30 min



Explicação dialogada e demonstração incremental

30 – 85 min



Oficina: uma das duas trilhas práticas

85 – 100  
min



Teste, commit, resumo, quiz e próxima aula

# Inferência e anotação se complementam

## INFERÊNCIA

O TypeScript deduz o tipo pelo valor inicial.

```
const titulo = "Memória";  
// TS sabe que é string
```

## ANOTAÇÃO

Nós declaramos o tipo explicitamente.

```
const ano: number = 2026;  
const pub: boolean = false;
```



O editor aponta incompatibilidades antes mesmo de você tentar executar o código.

# Unões restringem escolhas válidas

```
type Categoria =  
    "Relato" | "Lugar" | "Celebração";  
  
const categoria: Categoria = "Relato";
```

## O QUE É ISSO?

As **Union Types** permitem que uma variável aceite apenas um conjunto específico de valores fixos.

## ? PERGUNTA DE PREVISÃO

O que acontece se tentarmos atribuir o valor "Festa" à variável categoria?

# Uma interface descreve a forma de um objeto

```
interface Memoria {  
    id: number;  
    titulo: string;  
    comunidade: string;  
    categoria: Categoria;  
    ano?: number;  
}
```

## O CONTRATO

A **interface** funciona como um contrato: qualquer objeto que se diga uma "Memoria" deve seguir exatamente essa estrutura.

## ? PROPRIEDADE OPCIONAL

O símbolo **?** indica que o campo **ano** não é obrigatório. O objeto pode existir sem ele.

# Funções tipadas transformam dados com intenção clara

```
function criarLegenda(  
  memoria: Memoria  
): string {  
  return `${memoria.categoria} ·  
  ${memoria.comunidade}`;  
}
```

## CONTRATO DE ENTRADA E SAÍDA

Ao tipar o **parâmetro** e o **retorno**, garantimos que a função receba o objeto correto e devolva exatamente o que o app espera.

## 🔍 VERIFICAÇÃO

Quais erros o editor impediria se tentássemos acessar `memoria.data` dentro desta função?

# Uma coleção tipada mantém o contrato dos itens

```
const memorias: Memoria[] = [
  {
    id: 1,
    titulo: "Relato do Ancião",
    comunidade: "Itapecuru",
    categoria: "Relato"
  },
  {
    id: 2,
    titulo: "Casa de Farinha",
    comunidade: "Mirim",
    categoria: "Lugar"
  }
];
```

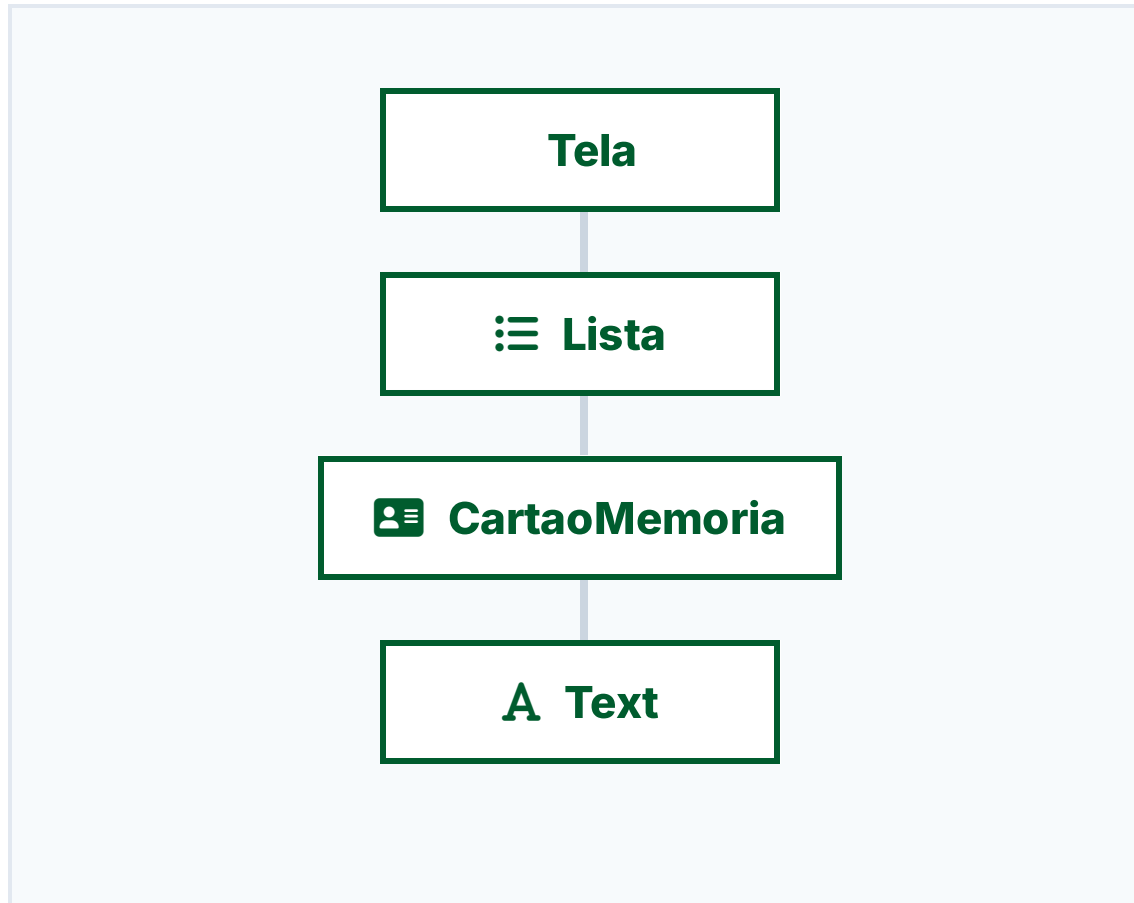
## O CONTRATO

Ao usar `Memoria[]`, garantimos que **todos** os itens da lista possuam as mesmas propriedades e tipos.

 **Nota:** Os dados acima são fictícios/simulados para fins didáticos.

 *Preparando o terreno para o uso do `.map()`.*

# Componentes dividem a tela em partes reutilizáveis



## REGRA DE OURO

O nome de todo componente deve começar com letra **Maiúscula** (**PascalCase**).

## O QUE ELE FAZ?

Um componente é uma função que retorna **JSX** (a estrutura visual que parece HTML).

🔄 *Crie uma vez, use em qualquer lugar.*

# Props levam dados do pai para o filho

```
type CartaoProps = {  
  memoria: Memoria;  
};  
  
function Cartao({ memoria }: CartaoProps) {  
  return (  
    <Text>{memoria.titulo}</Text>  
  );  
}
```

## → ENTRADA DE DADOS

As **props** são como os parâmetros de uma função: elas permitem que o componente pai envie informações para o filho.

 **Regra de Ouro:** Props são apenas para leitura. O componente filho nunca deve tentar modificar os dados que recebeu via props.

# Estado é a memória local do componente

```
const [idSelecionada, setIdSelecionada] =  
  useState<number | null>(null);
```

**1 VALOR ATUAL**  
idSelecionada guarda o dado agora.

**2 FUNÇÃO SETTER**  
setIdSelecionada altera o valor.

**3 ESTADO INICIAL**  
null é o ponto de partida.

 O estado permite que o componente "lembre" de escolhas do usuário entre uma renderização e outra.

# A interação atualiza o estado e a tela reage

```
<Pressable
  onPress={() =>
    setIdSelecionada(memoria.id)}
>
  <Text>Selecionar</Text>
</Pressable>
```

## A FUNÇÃO SETTER

A função `setIdSelecionada` não apenas muda o valor; ela avisa ao React que os dados mudaram e a tela precisa ser redesenhada.

## 🔄 O QUE MUDA?

`id = 10;`  
A tela ignora

`setId(10);`  
A tela reage

# .map() transforma dados em componentes

```
{memorias.map((memoria) => (  
  <Cartao  
    key={memoria.id}  
    memoria={memoria}  
  />  
))}
```

## ≡ RENDERIZAÇÃO DE LISTAS

O método **.map()** percorre o vetor de dados e cria uma instância do componente para cada item.

## 🔑 POR QUE USAR KEY?

A prop **key** ajuda o React a identificar quais itens mudaram, foram adicionados ou removidos, garantindo performance.

# Cinco erros que o editor ajuda a localizar

- ✗ Componente iniciado com letra minúscula
- ✗ Propriedade ( `prop` ) ausente ou com tipo errado
- ✗ Texto solto fora da tag `<Text>`
- ✗ Uso de `.map()` sem uma `key` estável
- ✗ Estado alterado diretamente (sem a função `setter` )

**Lembre-se:** O TypeScript ajuda a evitar erros de lógica durante a escrita, mas não substitui a validação de dados externos (APIs) em tempo de execução.

# Demonstração: O passo a passo técnico

● LIVE CODING

1



**MODELAGEM**

```
interface Memoria { ... }
```

2



**DADOS INICIAIS**

```
const m1: Memoria = { ... }
```

3



**COMPONENTIZAÇÃO**

```
function Cartao({ props })
```

4



**COLEÇÃO**

```
memorias.map(m => <Cartao />)
```

5



**INTERATIVIDADE**

```
const [sel, setSel] = useState()
```

# Escolha uma trilha prática em dupla



Trabalhem em duplas. Usem o arquivo inicial fornecido e não instalem novas bibliotecas.



## TRILHA A: MEMÓRIAS

Selecionar um cartão de memória simulada e revelar detalhes na tela.



## TRILHA B: EDA

Filtrar indicadores simulados que exigem atenção imediata.

# Trilha A: Memórias Quilombolas (Simuladas)

## OBJETIVO VISUAL

**MEMÓRIAS LOCAIS**

**Relato do A ancião**  
Relato · Itapecuru

**Casa de Farinha**  
Lugar · Mirim

Detalhes: História preservada desde 1950...



Restringir categorias com união de literais



Acrescentar uma propriedade opcional



Criar uma função tipada de legenda



Alternar a memória selecionada com useState



Testar e registrar evidência (screenshot)

**Atenção:** Os registros usados nesta oficina são fictícios e não representam comunidades reais. O foco é a prática técnica.

# Trilha B: EDA — Permanência Estudantil

## ANÁLISE DE DADOS

### Objetivo da Oficina

Criar um painel que filtre indicadores educacionais simulados que exigem atenção imediata da gestão.

### VISUALIZAÇÃO ESPERADA

Frequência: 72% [CRÍTICO]

Acesso AVA: 95% [ESTÁVEL]

✓ Restringir situação (Union Types)

✓ Criar função tipada de formatação

✓ Filtrar a coleção com `.filter()`

✓ Alternar o filtro com `useState`

✓ Testar e registrar evidência

***Nota Ética:** Os indicadores são fictícios, agregados e não contêm dados pessoais reais de estudantes.*

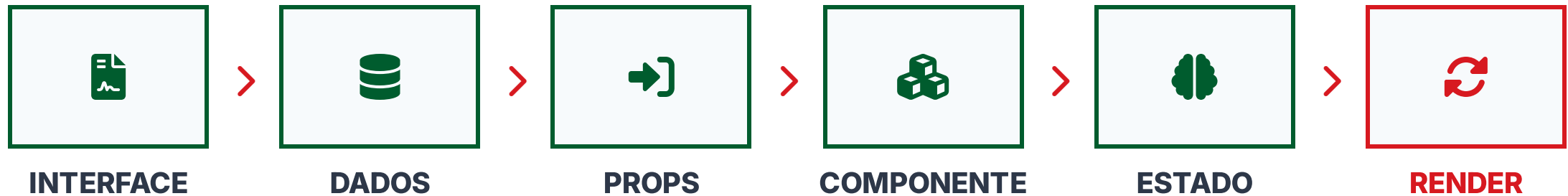
# Critério de pronto e depuração técnica

- ✔ O aplicativo abre no navegador/celular sem "tela vermelha" de erro
- ✔ O editor (VS Code) não apresenta erros de TypeScript no arquivo alterado
- ✔ A interação (toque/clique) altera a tela conforme o objetivo da trilha
- ✔ A dupla consegue explicar o uso de `interface`, `props` e `estado` no código
- ✔ Existe um commit realizado com uma mensagem clara e descritiva

## >\_ SUGESTÃO DE MENSAGEM

```
git commit -m "feat(aula-04): criar cartoes de memorias tipados"
```

## Resumo: O ciclo da reatividade móvel



### Voltando ao problema inicial...

Lembra do ano: "dois mil"? Agora, com a **Interface** definida, o editor impediria esse erro no momento da digitação, garantindo que apenas dados válidos cheguem aos seus **Componentes**.

# Desafio de Conhecimento (Parte 1)

1

**Qual construção descreve a forma de um objeto no TypeScript?**

A) interface    B) if    C) import

2

**As "Props" são dados de entrada ou a memória mutável do componente?**

A) Memória mutável    B) Dados de entrada

3

**Qual Hook do React adiciona memória local (estado) ao componente?**

A) useState    B) useMap    C) useProps

## Desafio de Conhecimento (Parte 2) e Saída

**4** Para que serve o método `.map()` neste aplicativo móvel?

**5** Por que é recomendado usar `memoria.id` como key em vez do índice do vetor?

**6** Verdadeiro ou Falso: O TypeScript valida automaticamente dados vindos de uma API externa em tempo de execução.

### **TICKET DE SAÍDA**

**"Em uma frase, explique a diferença entre `prop` e `estado` e cite a linha do seu código que mostra cada um."**

---

*Escreva no seu log de aula ou envie no canal da turma antes de sair do laboratório.*

# Próxima Aula: Interface e Acessibilidade



## EVOLUÇÃO VISUAL

*Transformando dados brutos em experiências reais.*

Na próxima aula, o mesmo componente ganhará uma interface móvel mais clara e acessível.



View & Text



Image



Pressable



StyleSheet



Flexbox



Acessibilidade

# Referências Oficiais (Parte 1)

■ MICROSOFT. **TypeScript Handbook: Everyday Types**. Disponível em: <https://www.typescriptlang.org/docs/handbook/2/everyday-types.html>. Acesso em: 18 ago. 2026.

■ MICROSOFT. **TypeScript Handbook: Object Types**. Disponível em: <https://www.typescriptlang.org/docs/handbook/2/objects.html>. Acesso em: 18 ago. 2026.

⚛ META. **Passing Props to a Component**. React Documentation. Disponível em: <https://react.dev/learn/passing-props-to-a-component>. Acesso em: 18 ago. 2026.

⚛ META. **State: A Component's Memory**. React Documentation. Disponível em: <https://react.dev/learn/state-a-components-memory>. Acesso em: 18 ago. 2026.

⚛ META. **useState**. React Reference. Disponível em: <https://react.dev/reference/react/useState>. Acesso em: 18 ago. 2026.

## Referências Oficiais (Parte 2)



**META.** *React Fundamentals*. React Native Docs. Disponível em:  
<https://reactnative.dev/docs/intro-react>



**META.** *Using TypeScript*. React Native Docs. Disponível em:  
<https://reactnative.dev/docs/typescript>



**EXPO.** *Using TypeScript*. Expo Documentation. Disponível em:  
<https://docs.expo.dev/guides/typescript/>



**EXPO.** *Create your first app — Run the app on mobile and web*. Expo Tutorial. Disponível em:  
<https://docs.expo.dev/tutorial/create-your-first-app/>